

nothing to do with it. It was an irrational, external, and entirely baseless need, but it was no less real.

Where did that need come from? It originated from the highly effective marketing campaigns employed by the database vendors at the time. They had managed to convince high-level executives that their corporate “data assets” needed protection, and that the database systems they offered were the ideal means of providing that protection.

We see the same kind of marketing campaigns today. The word “enterprise” and the notion of “Service-Oriented Architecture” have much more to do with marketing than with reality.

What *should* I have done in that long-ago scenario? I should have bolted an RDBMS on the side of the system and provided some narrow and safe data access channel to it, while maintaining the random access files in the core of the system. What *did* I do? I quit and became a consultant.

CONCLUSION

The organizational structure of data, the data model, is architecturally significant. The technologies and systems that move data on and off a rotating magnetic surface are not. Relational database systems that force the data to be organized into tables and accessed with SQL have much more to do with the latter than with the former. The data is significant. The database is a detail.

31

THE WEB IS A DETAIL



Were you a developer in the 1990s? Do you remember how the web changed everything? Do you remember how we looked at our old client-server architectures with disdain in the face of the shiny new technology of The Web?

Actually the web didn't change anything. Or, at least, it shouldn't have. The web is just the latest in a series of oscillations that our industry has gone through since the 1960s. These oscillations move back and forth between putting all the computer power in central servers and putting all computer power out at the terminals.

We've seen several of these oscillations just in the last decade or so since the web became prominent. At first we thought all the computer power would be in server farms, and the browsers would be stupid. Then we started putting applets in the browsers. But we didn't like that, so we moved dynamic content back to the servers.

But then we didn't like that, so we invented Web 2.0 and moved lots of processing back into the browser with Ajax and JavaScript. We went so far as to create whole huge applications written to execute in the browsers. And now we're all excited about pulling that JavaScript back into the server with Node.

(Sigh.)

THE ENDLESS PENDULUM

Of course, it would be incorrect to think that those oscillations started with the web. Before the web, there was client-server architecture. Before that, there were central minicomputers with arrays of dumb terminals. Before that, there were mainframes with smart green-screen terminals (that were very much analogous to modern-day browsers). Before that, there were computer rooms and punched cards ...

And so the story goes. We can't seem to figure out where we want the computer power. We go back and forth between centralizing it and distributing it. And, I imagine, those oscillations will continue for some time to come.

When you look at it in the overall scope of IT history, the web didn't change anything at all. The web was simply one of many oscillations in a struggle that began before most of us were born and will continue well after most of us have retired.

As architects, though, we have to look at the long term. Those oscillations are just short-term issues that we want to push away from the central core of our business rules.

Let me tell you the story of company Q. Company Q built a very popular personal finance system. It was a desktop app with a very useful GUI. I loved using it.

Then came the web. In its next release, company Q changed the GUI to look, and behave, like a browser. I was thunderstruck! What marketing genius decided that personal finance software, running on a desktop, should have the look and feel of a web browser?

Of course, I hated the new interface. Apparently everyone else did, too—because after a few releases, company Q gradually removed the browser-like feel and turned its personal finance system back into a regular desktop GUI.

Now imagine you were a software architect at Q. Imagine that some marketing

genius convinces upper management that the whole UI has to change to look more like the web. What do you do? Or, rather, what should you have done before this point to protect your application from that marketing genius?

You should have decoupled your business rules from your UI. I don't know whether the Q architects had done that. One day I'd love to hear their story. Had I been there at the time, I certainly would have lobbied very hard to isolate the business rules from the GUI, because you never know what the marketing geniuses will do next.

Now consider company A, which makes a lovely smartphone. Recently it released an upgraded version of its "operating system" (it's so strange that we can talk about the operating system inside a phone). Among other things, that "operating system" upgrade completely changed the look and feel of all the applications. Why? Some marketing genius said so, I suppose.

I'm not an expert on the software within that device, so I don't know if that change caused any significant difficulties for the programmers of the apps that run in company A's phone. I do hope the architects at A, and the architects of the apps, keep their UI and business rules isolated from each other, because there are always marketing geniuses out there just waiting to pounce on the next little bit of coupling you create.

THE UPSHOT

The upshot is simply this: The GUI is a detail. The web is a GUI. So the web is a detail. And, as an architect, you want to put details like that behind boundaries that keep them separate from your core business logic.

Think about it this way: *The WEB is an IO device*. In the 1960s, we learned the value of writing applications that were device independent. The motivation for that independence has not changed. The web is not an exception to that rule.

Or is it? The argument can be made that a GUI, like the web, is so unique and rich that it is absurd to pursue a device-independent architecture. When you think about the intricacies of JavaScript validation or drag-and-drop AJAX calls, or any of the plethora of other widgets and gadgets you can put on a web page, it's easy to argue that device independence is impractical.

To some extent, this is true. The interaction between the application and the GUI is

“chatty” in ways that are quite specific to the kind of GUI you have. The dance between a browser and a web application is different from the dance between a desktop GUI and its application. Trying to abstract out that dance, the way devices are abstracted out of UNIX, seems unlikely to be possible.

But another boundary between the UI and the application *can* be abstracted. The business logic can be thought of as a suite of use cases, each of which performs some function on behalf of a user. Each use case can be described based on the input data, the processing preformed, and the output data.

At some point in the dance between the UI and the application, the input data can be said to be complete, allowing the use case to be executed. Upon completion, the resultant data can be fed back into the dance between the UI and the application.

The complete input data and the resultant output data can be placed into data structures and used as the input values and output values for a process that executes the use case. With this approach, we can consider each use case to be operating the IO device of the UI in a device-independent manner.

CONCLUSION

This kind of abstraction is not easy, and it will likely take several iterations to get just right. But it is possible. And since the world is full of marketing geniuses, it's not hard to make the case that it's often very necessary.

FRAMEWORKS ARE DETAILS



Frameworks have become quite popular. Generally speaking, this is a good thing. There are many frameworks out there that are free, powerful, and useful.

However, frameworks are not architectures—though some try to be.

FRAMEWORK AUTHORS

Most framework authors offer their work for free because they want to be helpful to the community. They want to give back. This is laudable. However, regardless of their high-minded motives, those authors do not have *your* best interests at heart. They can't, because they don't know you, and they don't know your problems.